ДОКУМЕНТ ПРОЕКТНОЙ ДОКУМЕНТАЦИИ

Техническое задание

Проект: POTOK — веб-платформа публикации и просмотра видео
Формат: PDF

1) Цель проекта

Собрать функциональный MVP веб-приложения для публикации и просмотра видео с базовой социальной механикой: аккаунты, каналы, подписки, комментарии, уведомления и рекомендации.

Проект фокусируется на **бизнес-логике и данных**, без детальной визуальной проработки.

2) Технологический стек

- `Next.js` (App Router, TypeScript)
- `Supabase`:
- Auth (email/password, опционально OAuth)
- PostgreSQL
- Storage (видео, превью, аватары)
- RLS (Row Level Security)
- `Pusher` (или совместимый аналог) для real-time
- `Plyr` как единый видеоплеер (preview + watch page)

2.1 Архитектурные принципы (обязательные)

- **В БД хранятся только метаданные и ссылки/пути к медиа** (`video_url`, `thumbnail_url`, `avatar_url`), без бинарных файлов.
- **Компонентная вёрстка**: UI собирается из переиспользуемых компонентов (layout, cards, lists, forms, player, comments, notifications).
- Разделение на server/client компоненты и модули по доменам (`video`, `auth`, `comments`, `notifications`, `subscriptions`).

3) Границы MVP

Входит в MVP

- Регистрация/вход, профиль пользователя, страница канала
- Загрузка видео в Storage и публикация метаданных
- Категории, лента и фильтрация
- Страница просмотра видео `/video/[id]`
- Подписки и базовые рекомендации
- Комментарии с вложенностью + realtime обновление
- Уведомления (realtime + хранение в БД)
- Пагинация видео и комментариев

Не входит в MVP

- Финальный UI/дизайн-система/брендинг
- Сложные медиа-процессы (транскодинг, ABR, DRM)
- Продвинутая модерация/антиспам/антиабьюз
- Монетизация и рекламные инструменты
- **Live/стриминг** (прямые эфиры) — вне текущего объема

4) Роли и сценарии

Гость

- Просматривает публичные видео и категории
- Открывает страницу видео и читает комментарии

Авторизованный пользователь

- Управляет профилем
- Загружает видео
- Подписывается на каналы
- Ставит like/dislike
- Комментирует и отвечает на комментарии
- Получает уведомления

5) Функциональные требования

5.1 Аутентификация и пользователи

- Supabase Auth: email + password (OAuth как опция).
- Профиль пользователя: `username`, `avatar_url`, производные счетчики.

- Канал пользователя связан с аккаунтом (1:1).

Бизнес-правила

- `username` уникален (case-insensitive).
- Нельзя подписаться на самого себя.

5.2 Видео

- Форма загрузки: `title`, `description`, `category`, `tags`, `thumbnail`, `video`.
- Видео и изображения хранятся в Supabase Storage.
- Метаданные сохраняются в таблице `videos`.
- Уникальная страница просмотра: `/video/[id]`.

Обязательные поля для публикации

- `title`, `category_id`, `video_url`.

5.3 Категории

- Предзаполненный справочник `categories` (например: Музыка, Игры, Образование, Спорт).
- Выбор категории обязателен при публикации.
- На главной — фильтр по категориям.

5.4 Рекомендации (v1)

Рекомендации реализуются **серверной функцией** (Route Handler или Supabase Edge Function), с отдельным модулем логики.

Приоритеты подбора: 1. Видео каналов, на которые подписан пользователь. 2. Популярные видео за последние 7 дней (`views`). 3. Видео схожих категорий на основе истории просмотров пользователя.

Требование расширяемости

- Алгоритм должен быть изолирован в отдельном сервисе/функции для дальнейшей замены без переписывания UI.

5.5 Комментарии (realtime)

- Под видео отображается список комментариев и ответов.
- Иерархия: комментарий → ответы (через `parent_id`).
- Создание комментария отправляет событие в Pusher.
- Клиенты получают события и обновляют UI без перезагрузки.

5.6 Уведомления

- Канал доставки: `user-{userId}`.
- Типы:
- новый комментарий к моему видео,
- ответ на мой комментарий,
- новое видео от канала, на который я подписан.
- Все уведомления пишутся в `notifications`.
- Поддержка статуса прочтения `is_read`.

5.7 Сайдбар

- Разделы: Главная, Подписки, Библиотека, Категории.
- Состояние свернуто/развернуто хранится локально (localStorage/контекст).
- При сворачивании контентная область расширяется.

5.8 Plyr

Единый компонент плеера используется в 2 режимах:

- `preview` (мини-проигрывание/hover/модалка),
- `watch` (основная страница видео).

Компонент принимает URL видео и режим работы.

5.9 Модерация, жалобы и роли

- Пользователь может отправить жалобу на видео, комментарий или канал с выбором причины из фиксированного списка и пояснением.
- Очередь жалоб обрабатывают роли `moderator` и `admin` (панель `/admin`): фильтры по статусу и причине, поиск по тексту жалобы, закрытие с заметкой, удаление комментария или скрытие видео, бан пользователя с указанием срока и причины.
- Автор видео может удалять комментарии под своими роликами и отмечать комментарии «сердцем автора» (неограниченное число комментариев).
- Ответы на комментарии — **один уровень** (ответ только к корневому комментарию); при ответе адресату приходит уведомление в `notifications`.
- Рекомендации на главной и в боковой панели просмотра: эвристика по просмотрам, лайкам, свежести, подпискам, истории просмотров и лайков, категории и автору.

6) Схема данных (Supabase/PostgreSQL)

Ниже минимально необходимая структура.

`users` (расширение профиля)

- `id uuid pk` (ссылка на `auth.users.id`)
- `username text unique not null`
- `avatar_url text null`
- `subscribers_count int default 0`
- `created_at timestamptz default now()`

`categories`

- `id uuid pk`
- `name text unique not null`
- `slug text unique not null`

`videos`

- `id uuid pk`
- `user_id uuid not null`
- `title text not null`
- `description text null`
- `category_id uuid not null`
- `tags text[] default '{}'`
- `video_url text not null`
- `thumbnail_url text null`
- `views int default 0`
- `created_at timestamptz default now()`

`subscriptions`

- `subscriber_id uuid not null`
- `channel_id uuid not null`
- `created_at timestamptz default now()`
- `primary key (subscriber_id, channel_id)`

`likes`

- `user_id uuid not null`
- `video_id uuid not null`
- `type text check (type in ('like','dislike'))`
- `created_at timestamptz default now()`
- `primary key (user_id, video_id)`

`comments`

- `id uuid pk`

- `video_id uuid not null`
- `user_id uuid not null`
- `parent_id uuid null`
- `content text not null`
- `created_at timestamptz default now()`

`notifications`

- `id uuid pk`
- `user_id uuid not null`
- `type text not null`
- `data jsonb not null`
- `is_read boolean default false`
- `created_at timestamptz default now()`

`reports` (жалобы)

- `reporter_id`, `target_type` (video | comment | channel), `target_id`, `reason_code`, `details`, `status`, `resolved_at`, `resolved_by`, `resolution_note`.

`comment_author_hearts`

- `comment_id`, `video_owner_id` — отметка автора канала на комментарии.

В профиле `users` дополнительно: `role` (user | moderator | admin), `banned_until`, `ban_reason_code`.

7) Индексы и ограничения (минимум)

- Индексы:
- `videos(created_at desc)`
- `videos(category_id, created_at desc)`
- `videos(views desc)`
- `comments(video_id, created_at asc)`
- `notifications(user_id, created_at desc)`
- Ограничения:
- `subscriptions.subscriber_id != subscriptions.channel_id`
- `comments.parent_id` может ссылаться только на комментарий того же `video_id` (проверка в сервисе/триггере)

8) RLS-политики (требуемый минимум)

- Публичное чтение: `categories`, публичные поля `videos`, `comments`.
- Пользователь может:

- редактировать только свой профиль,
- добавлять/удалять только свои подписки,
- добавлять и удалять только свои комментарии,
- создавать/редактировать/удалять только свои видео,
- читать/обновлять (is_read) только свои уведомления.

9) API/сервисные контракты (минимум)

Примерная структура endpoint-слоя:

- `POST /api/videos/upload` — публикация видео и метаданных.
- `GET /api/videos/recommendations` — рекомендации v1.
- `GET /api/videos/:id/comments` + пагинация.
- `POST /api/videos/:id/comments` — новый комментарий.
- `POST /api/videos/:id/like` — like/dislike.
- `POST /api/channels/:id/subscribe` — подписка/отписка.
- `GET /api/notifications` и `PATCH /api/notifications/:id/read`.

10) Real-time события (Pusher)

Минимальный набор событий:

- `comment.created`
- `comment.replied`
- `notification.created`
- `video.created` (для подписчиков)

Формат payload должен содержать `id`, `type`, `created_at`, и данные сущности.

11) Пагинация

- Видео: cursor-based (по `created_at`, `id`) или page+limit (для MVP допустимо).
- Комментарии: page+limit, ответы подгружаются отдельно или вместе (ограниченно).

12) Нефункциональные требования

- Чистая модульная структура проекта.
- Разделение server/client логики.
- Обработка ошибок загрузки/доступа/валидации.
- Базовая наблюдаемость: логирование ключевых ошибок.
- Компонентная организация UI с минимизацией дублирования и явными контрактами пропсов.

13) Критерии готовности MVP (Definition of Done)

- Пользователь может зарегистрироваться, войти, заполнить профиль.
- Пользователь может загрузить видео и открыть его по `/video/[id]`.
- Лента показывает рекомендации по правилам v1.
- Комментарии и ответы обновляются в realtime у нескольких клиентов.
- Уведомления создаются, приходят через realtime и сохраняются в БД.
- Работают базовые RLS-политики, пользователь не видит/не изменяет чужие приватные данные.
- Есть пагинация видео и комментариев.

14) Этапы реализации

1. Инициализация проекта и env-конфигурации (`Next.js + Supabase + Pusher`). 2. Миграции БД, индексы, RLS. 3. Auth + профиль + канал. 4. Загрузка и публикация видео, страница просмотра, Plyr. 5. Категории, главная лента, базовые рекомендации. 6. Подписки, likes/dislikes, библиотека. 7. Комментарии + realtime. 8. Уведомления + realtime виджет. 9. Пагинация, обработка ошибок, стабилизация.

15) Явные ограничения разработки

- Не делать детальный дизайн и кастомные темы на данном этапе.
- Не подключать тяжелые UI-фреймворки ради внешнего вида.
- Фокус только на функциональности и корректной архитектуре.
- Не реализовывать live-эфиры/стриминг и связанные с ними сущности в БД/API.

Документ сформирован автоматически из репозитория проекта. При расхождении с актуальной разработкой ориентируйтесь на кодовую базу и согласованные изменения ТЗ.